Université de Yaoundé I

INF4067 : UML et Design Patterns | 2026-2027

Les principes SOLID

Modèles UML, implémentations Java (versions bad et good) et tests

Étudiant : TIOZANG NGOUANA NELSON THIBAUT __ 23V2820

Enseignant : Valéry MONTHE

Date : novembre 2026

Code source (GitHub) : https://github.com/Nelthi/INF_4067

1. Introduction et organisation du projet

Ce rapport présente la réalisation des travaux pratiques sur les principes SOLID. Pour chaque principe du cours (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion), ainsi que pour l'exemple d'introduction (moyens de paiement) et l'exemple de cohésion et couplage, nous avons :

- réalisé le modèle UML (diagramme de classes) de la version mauvaise (bad) et de la version bonne (good) ;
- implémenté les deux versions en Java ;
- testé la version propre (good) ;
- documenté le projet dans un fichier README.

Le code complet est disponible sur GitHub : https://github.com/Nelthi/INF_4067

Le dépôt est organisé ainsi :

Emplacement	Contenu
code/<sujet>/bad	Version qui viole le principe (package <sujet>.bad)
code/<sujet>/good	Version refactorisée (package <sujet>.good)
docs/uml	Diagrammes de classes (sources PlantUML et images PNG)
README.md	Description du projet, prérequis, exemples

2. Rappel des principes

	Principe	Énoncé
S	Single Responsibility	Un module ne doit avoir qu'une seule raison de changer.
O	Open/Closed	Les entités sont ouvertes à l'extension et fermées à la modification.
L	Liskov Substitution	Un sous-type doit pouvoir remplacer son type de base sans comportement non désiré.
I	Interface Segregation	Un client ne doit pas dépendre de méthodes qu'il n'utilise pas ; préférer de petites interfaces.
D	Dependency Inversion	Les modules de haut niveau ne dépendent pas de ceux de bas niveau : tous dépendent d'abstractions.

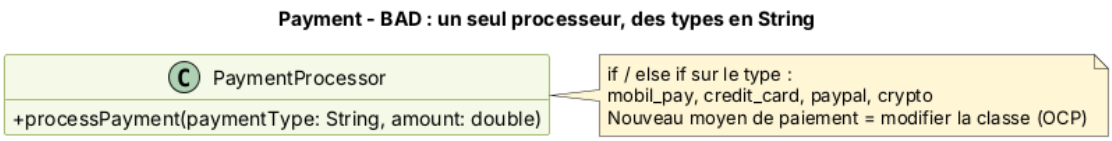
3. Exemple d'introduction : moyens de paiement

Idée directrice : Introduction aux principes (OCP, DIP, SRP).

3.1 Version mauvaise (bad)

La classe PaymentProcessor traite tous les moyens de paiement (mobile, carte de crédit, PayPal, crypto-monnaie) avec une cascade de if / else if sur une chaîne de caractères.

Ajouter un nouveau moyen de paiement oblige à modifier PaymentProcessor (rigidité), et une faute de frappe dans le type est ignorée sans erreur (fragilité). Le code dépend directement des détails de chaque mode de paiement.



Modèle UML de la version bad (payment)

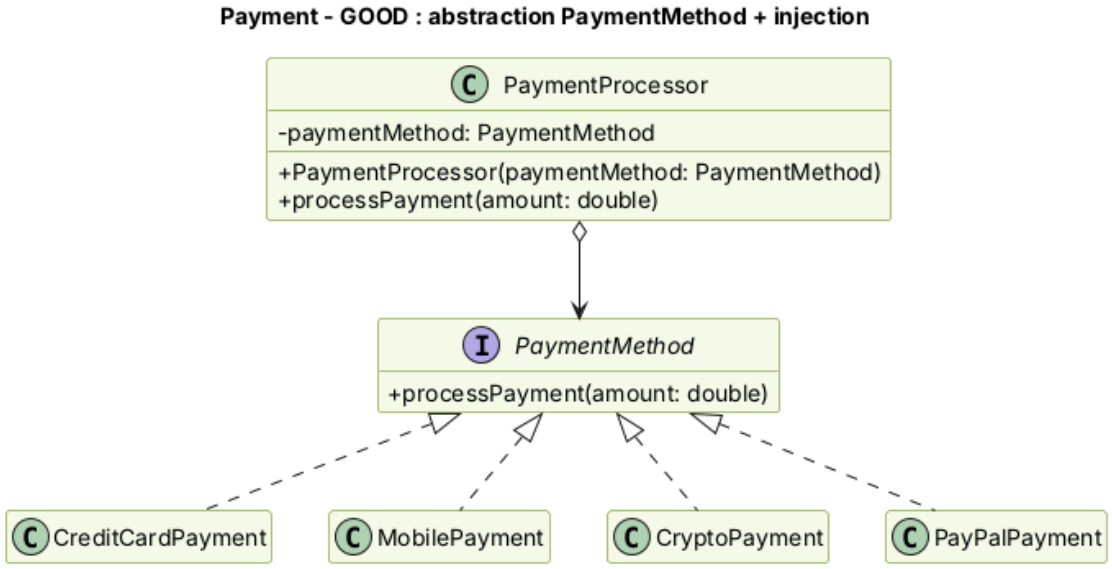
Exécution de payment.bad.Main

```
Processing mobile payment: 1500.0 CFA
Processing credit card payment: 6500.0 CFA
Processing PayPal payment: 4500.0 CFA
Processing crypto payment: 8500.0 CFA
```

3.2 Version bonne (good)

Une interface PaymentMethod définit le contrat processPayment(amount). Chaque moyen de paiement (CreditCardPayment, MobilePayment, CryptoPayment, PayPalPayment) l'implémente.

PaymentProcessor reçoit un PaymentMethod par son constructeur (injection de dépendance) : il ne connaît plus aucun détail concret et un nouveau moyen de paiement s'ajoute par une simple nouvelle classe.



Modèle UML de la version good (payment)

Exécution de payment.good.Main

```
Processing mobile payment: 1500.0 CFA
Processing credit card payment: 6500.0 CFA
Processing PayPal payment: 4500.0 CFA
Processing crypto payment: 8500.0 CFA
```

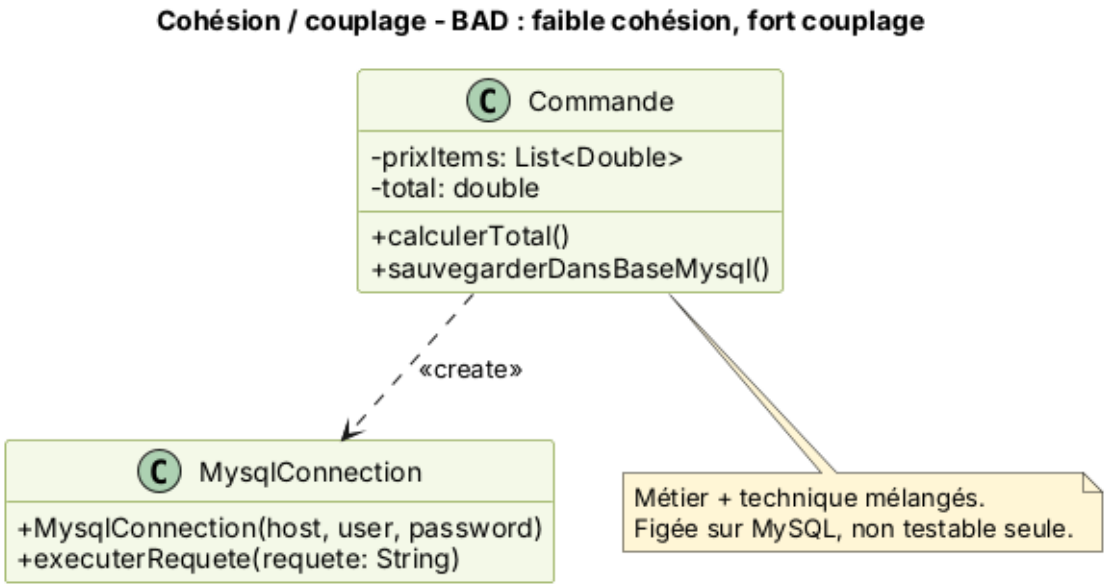
4. Cohésion et couplage : Commande et base de données

Idée directrice : Forte cohésion, faible couplage.

4.1 Version mauvaise (bad)

La classe Commande mélange la logique métier (calculerTotal) et un détail technique (sauvegarderDansBaseMysql qui ouvre une connexion MySQL).

Faible cohésion : deux responsabilités sans rapport. Fort couplage : la classe crée elle-même MysqlConnection, elle est figée sur MySQL et ne peut pas être testée sans base de données.



Modèle UML de la version bad (cohesion)

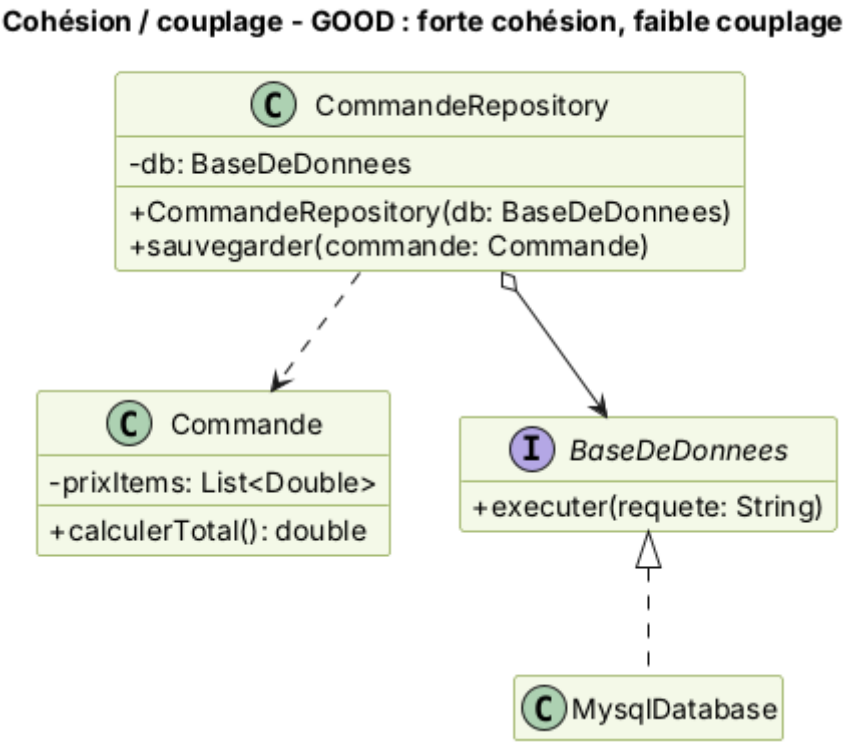
Exécution de cohesion.bad.Main

```
Connexion à MySQL sur localhost (utilisateur root)
Exécution MySQL : INSERT INTO commandes (total) VALUES (5000.0)
```

4.2 Version bonne (good)

Commande ne gère que les données et le calcul du total. CommandeRepository gère uniquement la persistance et dépend de l'interface BaseDeDonnees, implémentée par MysqlDatabase (ou toute autre base).

Chaque classe a une seule raison de changer (cohésion) et le dépôt est découplé du SGBD choisi (couplage faible).



Modèle UML de la version good (cohesion)

```
Exécution de cohesion.good.Main
Exécution MySQL : INSERT INTO commandes (total) VALUES (5000.0)
```

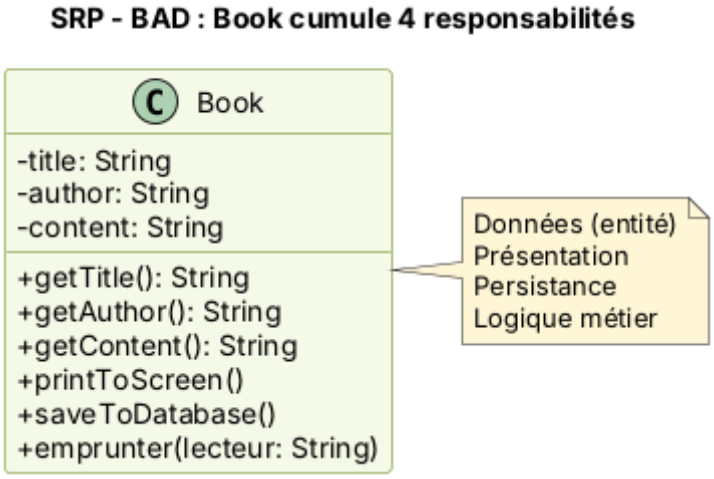
5. S : Single Responsibility Principle

Idée directrice : Une classe = une seule raison de changer.

5.1 Version mauvaise (bad)

La classe Book cumule quatre responsabilités : les données du livre (entité), l'affichage (présentation), la sauvegarde (persistance) et l'emprunt (logique métier).

Modifier l'un de ces aspects force à modifier la même classe : fort couplage et faible maintenabilité.



Modèle UML de la version bad (srp)

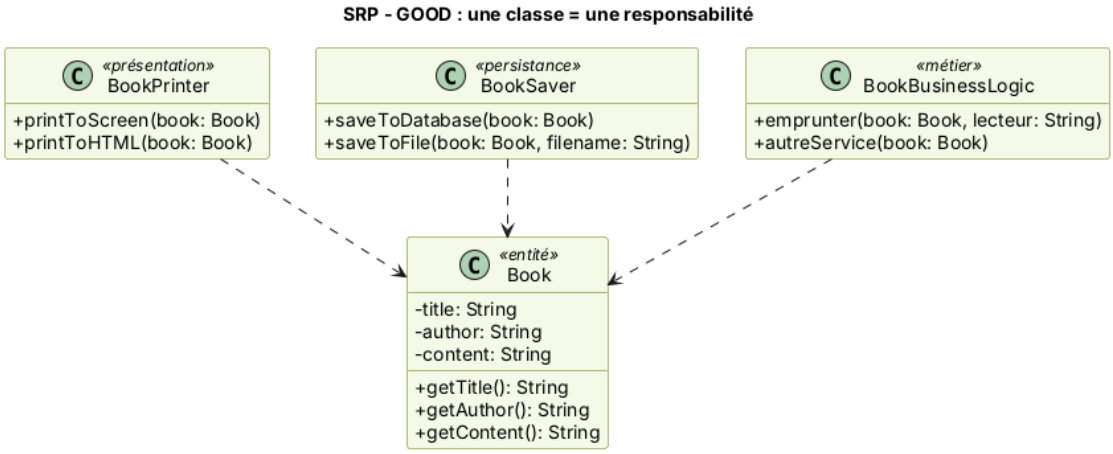
Exécution de srp.bad.Main

```
Titre: Les principes SOLID
Auteur: Etudiants de M1-GL
Contenu: Révision des principes SOLID
Sauvegarde du livre 'Les principes SOLID' en base de données...
Emprunt du livre 'Les principes SOLID' par Marcial
```

5.2 Version bonne (good)

Les quatre responsabilités sont séparées : Book (entité), BookPrinter (présentation), BookSaver (persistance) et BookBusinessLogic (logique métier).

On peut ajouter un format d'affichage ou un mode de sauvegarde sans toucher à Book.



Modèle UML de la version good (srp)

Exécution de srp.good.Main

```
===Print to Screen===
Titre: Les principes SOLID
Auteur: Etudiants de M1-GL
Contenu: Révision des principes SOLID
===Print to HTML===
<h1>Les principes SOLID</h1>
<h2>Par Etudiants de M1-GL</h2>
<p>Révision des principes SOLID</p>
Sauvegarde de 'Les principes SOLID' en base de données...
Emprunt du livre 'Les principes SOLID' par Marcial
```

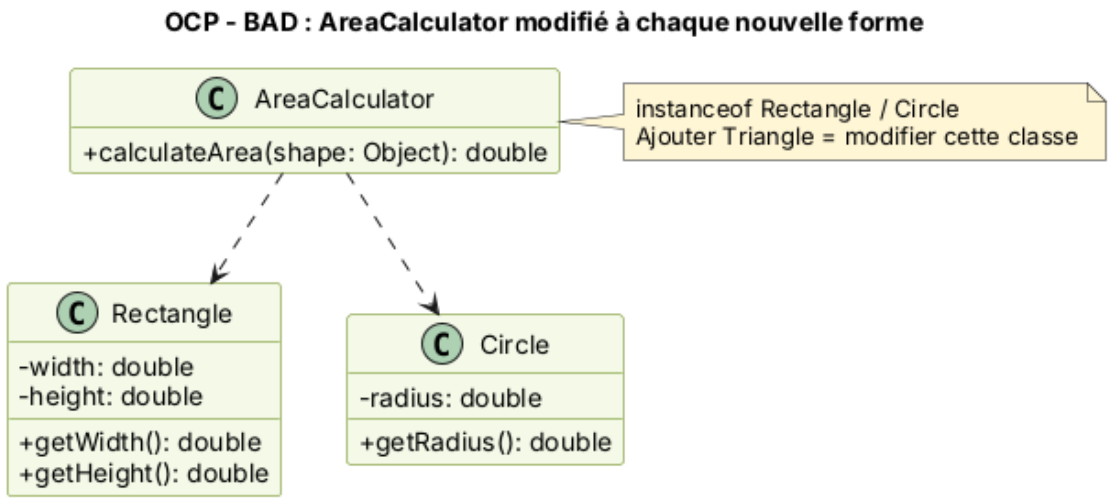
6. O : Open/Closed Principle

Idée directrice : Ouvert à l'extension, fermé à la modification.

6.1 Version mauvaise (bad)

AreaCalculator.calculateArea(Object) teste le type de la forme avec instanceof (Rectangle, Circle).

Pour ajouter une forme (par exemple un triangle), il faut rouvrir et modifier AreaCalculator : le code déjà testé et livré est remis en cause à chaque évolution.



Modèle UML de la version bad (ocp)

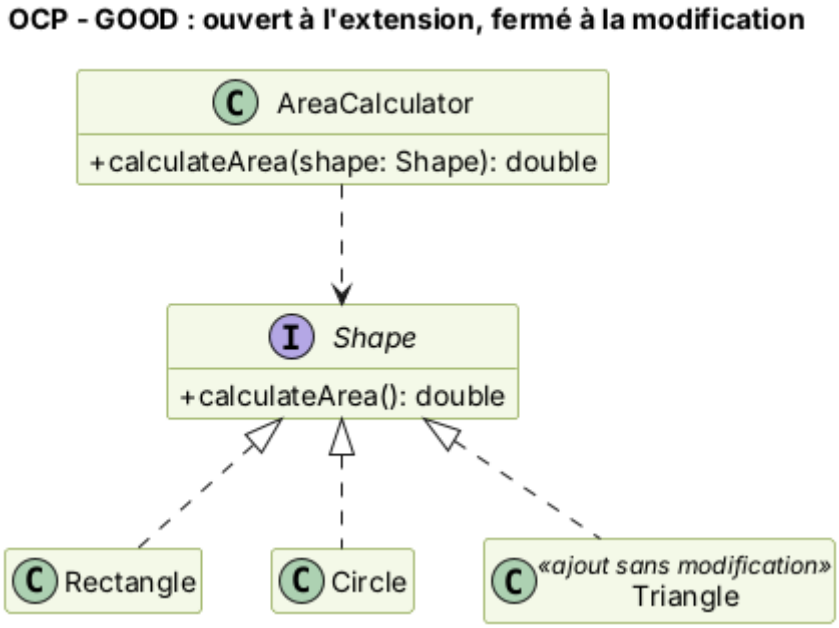
Exécution de ocp.bad.Main

```
Aire du rectangle = 12.0
Aire du cercle    = 12.566370614359172
Nouvelle forme non supportée sans modifier la classe : Unknown shape
```

6.2 Version bonne (good)

Une interface Shape déclare calculateArea(). Rectangle, Circle et Triangle l'implémentent, et AreaCalculator délègue simplement à shape.calculateArea().

Triangle a été ajouté sans modifier ni Shape ni AreaCalculator : l'extensibilité se traduit par de l'ajout de code uniquement, grâce à l'abstraction et au polymorphisme.



Modèle UML de la version good (ocp)

Exécution de ocp.good.Main

```
Aire du rectangle = 12.0
Aire du cercle    = 12.566370614359172
Aire du triangle  = 6.0
```

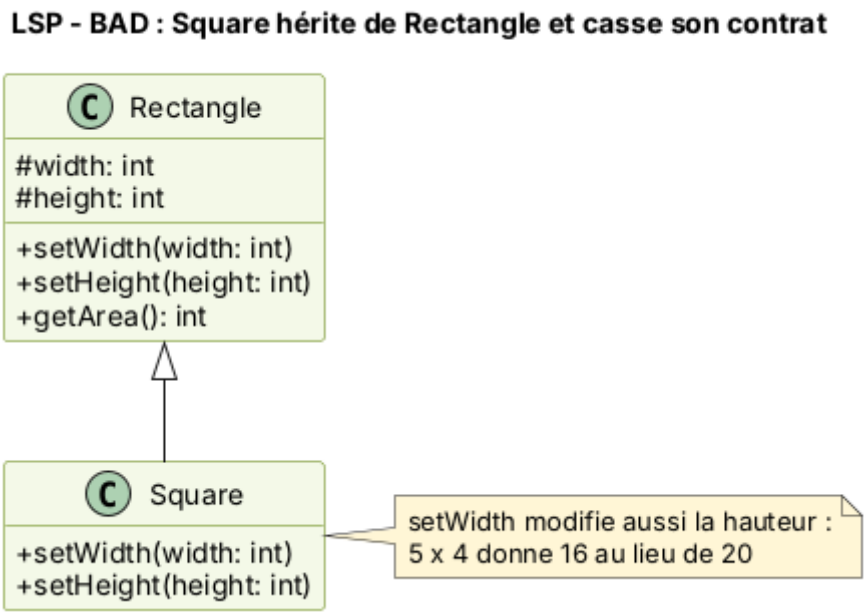
7. L : Liskov Substitution Principle

Idée directrice : Les sous-classes doivent être substituables à leur classe de base.

7.1 Version mauvaise (bad)

Square hérite de Rectangle et redéfinit setWidth et setHeight pour garder les deux côtés égaux.

Le carré ne respecte pas le contrat de Rectangle : après setWidth(5) puis setHeight(4), on attend une aire de 20 mais un Square donne 16. Remplacer un Rectangle par un Square change le comportement du programme.



Modèle UML de la version bad (lsp)

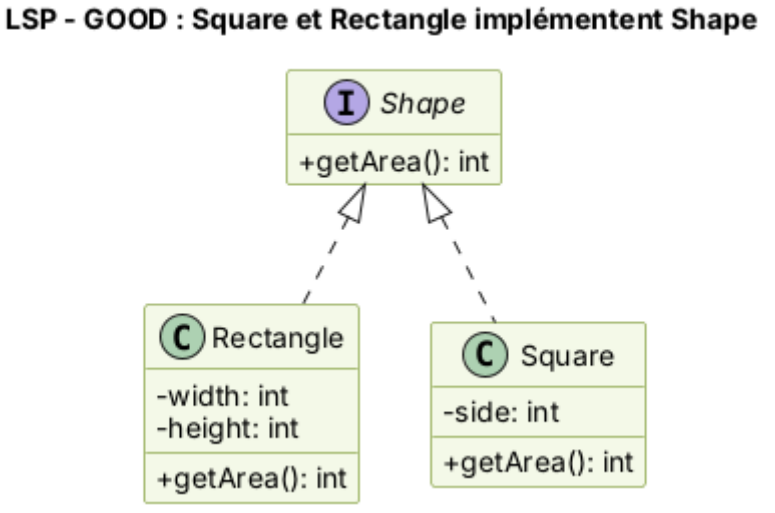
Exécution de lsp.bad.Main

```
Aire du rectangle = 20
Aire du carré (vu comme un Rectangle) = 16  <-- résultat inattendu
```

7.2 Version bonne (good)

Square n'hérite plus de Rectangle. Les deux classes implémentent une interface Shape qui ne partage que le contrat « calculer une aire ».

Toutes les implémentations de Shape sont interchangeables sans surprise.



Modèle UML de la version good (lsp)

Exécution de lsp.good.Main

```
Square Area : 9
Rectangle Area : 12
```

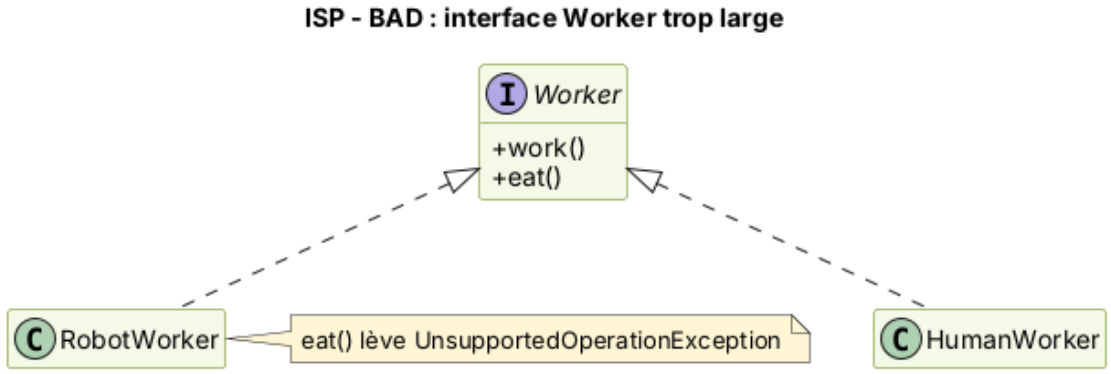
8. I : Interface Segregation Principle

Idée directrice : Préférer des interfaces petites et ciblées.

8.1 Version mauvaise (bad)

L'interface Worker déclare work() et eat(). RobotWorker est obligé d'implémenter eat(), qui n'a pas de sens pour un robot, et lève une UnsupportedOperationException.

Le client dépend d'une méthode qu'il n'utilise pas, et l'erreur n'apparaît qu'à l'exécution.



Modèle UML de la version bad (isp)

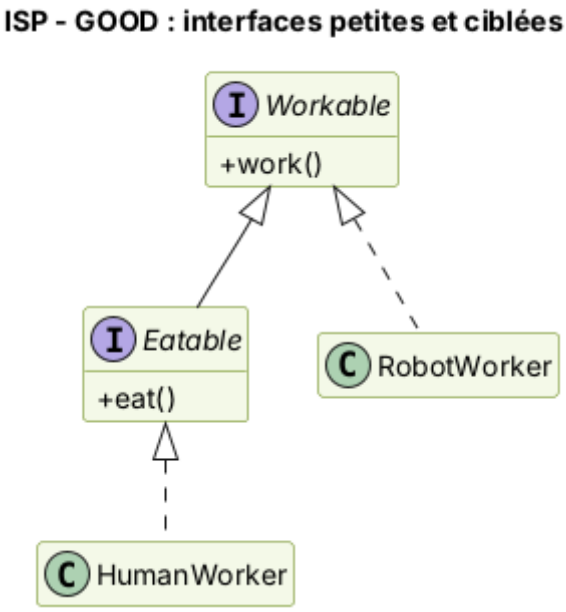
Exécution de isp.bad.Main

```
Les humains travaillent
Les humains mangent
Les robots travaillent sans fatigue
Erreur à l'exécution : Les robots ne mangent pas
```

8.2 Version bonne (good)

L'interface est découpée : Workable (work) et Eatable qui étend Workable (eat). HumanWorker implémente Eatable, RobotWorker implémente seulement Workable.

Chaque classe ne dépend que de ce dont elle a besoin ; appeler eat() sur un robot ne compile plus.



Modèle UML de la version good (isp)

Exécution de isp.good.Main

```
Les humains travaillent
Les humains mangent
Les robots travaillent sans fatigue
```

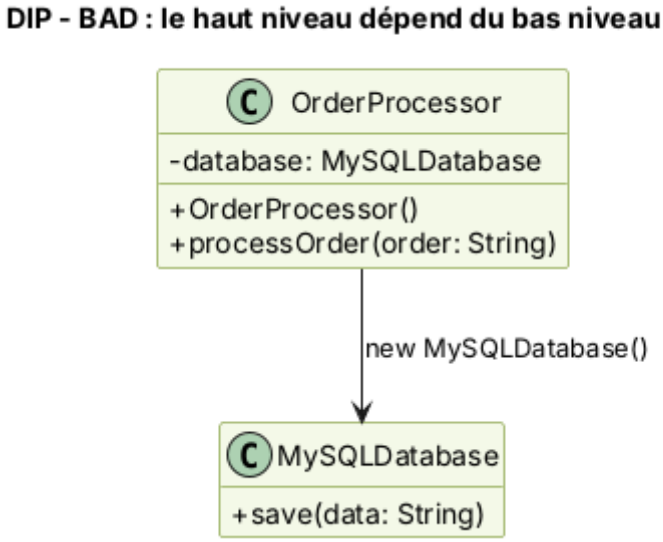
9. D : Dependency Inversion Principle

Idée directrice : Dépendre d'abstractions, pas de classes concrètes.

9.1 Version mauvaise (bad)

OrderProcessor (module de haut niveau, métier) crée directement une MySQLDatabase (module de bas niveau) dans son constructeur.

Il est impossible de changer de base de données ou de tester OrderProcessor sans MySQL, et la réutilisation oblige à embarquer la classe concrète.



Modèle UML de la version bad (dip)

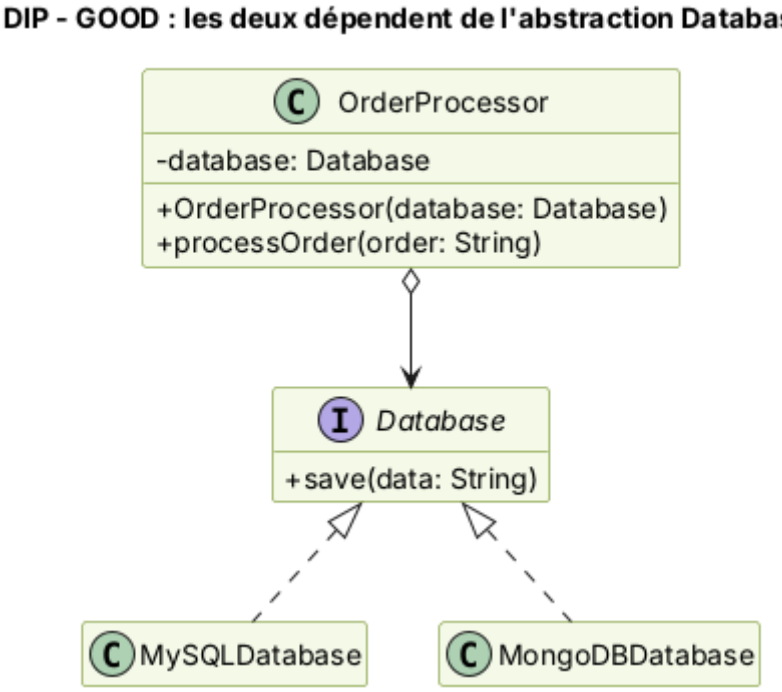
Exécution de dip.bad.Main

```
Saving to MySQL: Données à sauvegarder
```

9.2 Version bonne (good)

OrderProcessor dépend de l'interface Database, reçue par injection via le constructeur. MySQLDatabase et MongoDBDatabase implémentent cette interface.

Le haut niveau et le bas niveau dépendent tous deux de l'abstraction : on change de base en changeant seulement l'objet injecté.



Modèle UML de la version good (dip)

Exécution de dip.good.Main

```
Saving to MySQL: Données à sauvegarder
Saving to MongoDB: Données à sauvegarder
```

10. Synthèse et résultats des tests

Sujet	Problème de la version bad	Apport de la version good
Paiement	if / else sur des chaînes ; modification à chaque nouveau moyen	Interface PaymentMethod + injection
Cohésion	Métier et SQL mélangés ; figé sur MySQL	Commande, CommandeRepository, BaseDeDonnees
SRP	Book cumule 4 responsabilités	4 classes à responsabilité unique
OCP	instanceof ; modifier la classe pour chaque forme	Interface Shape ; Triangle ajouté sans modification
LSP	Square donne 16 au lieu de 20	Square et Rectangle implémentent Shape
ISP	RobotWorker lève une exception sur eat()	Workable et Eatable
DIP	new MySQLDatabase() dans le métier	Dépendance à l'interface Database

11. Conclusion

Les exemples montrent que les cinq principes convergent vers le même objectif : maîtriser les dépendances pour obtenir un code robuste, extensible et réutilisable. Dans chaque cas, la version good repose sur les mêmes outils : des abstractions (interfaces), le polymorphisme et l'injection de dépendances, qui rendent aussi le code plus facile à tester.

Dépôt du projet : https://github.com/Nelthi/INF_4067